فصل ۲: HTML

پیش‌نیاز: فصل ۱ (نقشه راه) و آماده بودن ابزارها (مرورگر، VS Code، Live Server). **این فصل سه سطح دارد:** مقدماتی، متوسط، پیشرفته. در پایان هر سطح تمرین، «آیا بلدم؟» و پروژه آمده است.

نقشه‌ی فصل

بخش	موضوع	اهمیت
۲.۱	HTML چیست؟	[ضروری]
۲.۲	ساختار سند (DOCTYPE، html، head، body، title، meta)	[ضروری]
۲.۳	Heading، پاراگراف و متن	[ضروری]
۲.۴	لینک‌ها	[ضروری]
۲.۵	تصاویر	[ضروری]
۲.۶	لیست‌ها	[ضروری]
۲.۷	دکمه‌ها	[ضروری]
۲.۸	کامنت‌ها	[ضروری]
۲.۹	Attribute، class و id	[ضروری]
۲.۱۰	Block، Inline، div و span	[ضروری]
۲.۱۱ تا ۲.۱۵	HTML متوسط: Semantic، فرم‌ها، Validation، جدول	[ضروری] / [مفید حرفه‌ای]
۲.۱۶ تا ۲.۲۱	HTML پیشرفته: Accessibility، SEO، مدیا، عناصر مدرن، Performance، چک‌لیست	[ضروری] تا [پیشرفته/ اختیاری]

بخش A: HTML مقدماتی

۲.۱ HTML چیست؟ [ضروری]

مفهوم. HTML مخفف HyperText Markup Language است. یک **زبان نشانه‌گذاری** است، یعنی با آن به مرورگر می‌گوییم هر تکه از محتوا «چیست»: عنوان، پاراگراف، لینک، تصویر، فرم و غیره.

HTML **زبان برنامه‌نویسی نیست.** متغیر، شرط و حلقه ندارد. منطق را JavaScript اضافه می‌کند.

ساختار یک المان (Element):

```
<p class="intro">سلام دنیا</p>
```

```
<p class="intro">   سلام دنیا   </p>
|  |              |           |
|  attribute      content     closing tag
opening tag
```

- **تگ باز** (`<p>`) و **تگ بسته** (`</p>`) و **محتوا** در بینشان، روی هم یک **المان** می‌سازند.
- بعضی المان‌ها محتوا ندارند و تگ بسته هم ندارند. به آن‌ها **Void Element** می‌گویند: `<br>` ، `<hr>` ، `<link>` ، `<meta>` ، `<input>` ، `<img>` .
- المان‌ها می‌توانند داخل هم قرار بگیرند (**Nesting**) و درخت DOM همین‌طور ساخته می‌شود:

```
<p>این یک <strong>مهم</strong> است.</p>
```

اشتباهات رایج

- فراموش کردن تگ بسته. HTML بسیار «بخشنده» است و خطا نمی‌دهد، ولی نتیجه‌ی نمایش خراب می‌شود و پیدا کردن علتش سخت است.
- بستن نادرست تو در تو: `<p><strong>متن</p></strong>` غلط است. هر تگی که آخر باز شده، باید اول بسته شود.
- اعتماد به اینکه «در مرورگر درست نشان می‌دهد» یعنی کد درست است. از اعتبارسنج رسمی (validator.w3.org) استفاده کن.

> **تمرین کوچک:** المان‌های این قطعه را شمار و بگو کدام Void است و کدام در کدام تو در تو شده:
> `<p>سلام <em>دنیا</em><br>خوش آمدی</p>`

۲.۲ ساختار یک سند HTML [ضروری]

مفهوم. هر صفحه‌ی HTML یک قالب ثابت دارد. مرورگر انتظار دارد این اجزا را ببیند:

```
 <!DOCTYPE html>
<html lang="fa" dir="rtl">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>صفحه‌ی اول من</title>
</head>
<body>
  <h1>سلام</h1>
  <p>این اولین صفحه‌ی من است.</p>
</body>
</html>
```

توضیح خط‌به‌خط:

بخش	توضیح
`<!DOCTYPE html>`	به مرورگر می‌گوید سند HTML5 است. بدون آن، مرورگر در «Quirks Mode» رندر می‌کند و رفتارهای قدیمی و غیرمنتظره می‌بیند. تگ نیست، یک اعلان است.
`<html lang="fa" dir="rtl">`	ریشه‌ی سند. `lang` زبان صفحه را مشخص می‌کند (برای screen reader، ترجمه‌ی خودکار و SEO) و `dir` جهت متن را.
`<head>`	اطلاعات **درباره‌ی** صفحه (نمایش داده نمی‌شود).
`<meta charset="UTF-8">`	کدگذاری کاراکترها. بدون آن ممکن است فارسی به شکل کاراکترهای به‌هم‌ریخته دیده شود. باید در ۱۰۲۴ بایت اول فایل باشد، پس اولین مورد داخل `head` بگذار.
`<meta name="viewport" ...>`	برای موبایل لازم است. بدون آن صفحه روی موبایل مثل دسکتاپ کوچک‌شده نمایش داده می‌شود. جزئیات در بخش پیشرفته.
`<title>`	عنوان تب مرورگر، Bookmark و نتایج جستجو.
`<body>`	هر چیزی که کاربر می‌بیند.

تفاوت `head` و `body`: `head` برای مرورگر و موتور جستجو است، `body` برای کاربر. تنها چیز داخل `head` که کاربر مستقیم می‌بیند، `title` در تب است.

اشتباهات رایج

- فراموش کردن `<!DOCTYPE html>` یا `charset`.
- گذاشتن محتوای قابل‌نمایش در `head`.
- ننوشتن `lang`. (برای فارسی `fa`، برای انگلیسی `en`.)
- مقدار یکسان `title` در همه‌ی صفحه‌های یک سایت.

> **تمرین کوچک:** یک فایل `index.html` با ساختار بالا بساز. `title` را عوض کن و تغییرش را در تب مرورگر ببین. بعد `charset` را حذف کن و ببین چه می‌شود (ممکن است در بعضی مرورگرها فارسی خراب نشود، چون حدس می‌زنند، ولی به آن تکیه نکن).

۲.۳ Heading، پاراگراف و متن [ضروری]

Heading: `h1` تا `h6` برای عنوان‌گذاری **سلسله‌مراتبی**. `h1` مهم‌ترین، `h6` کم‌اهمیت‌ترین.

```
<h1>آموزش وب</h1>
<h2>HTML</h2>
<h3>تگ‌های متنی</h3>
<h2>CSS</h2>
```

پاراگراف: `<p>`.

قوانین مهم:

- Heading برای **معنا** است، نه برای بزرگ‌کردن فونت. اندازه را با CSS تعیین می‌کنیم.
- سلسله‌مراتب را نشکن: بعد از `h2` مستقیم `h4` نگذار.
- توصیه‌ی رایج: در هر صفحه یک `h1` که موضوع اصلی صفحه را بگوید.

فضای خالی (Whitespace) در HTML: چند Space و Enter پشت‌سرهم، فقط یک فاصله نمایش داده می‌شوند:

```
<p>سلام      به
   همه</p>
```

نتیجه: «سلام به همه» (در یک خط، با یک فاصله). برای جدا کردن بخش‌ها از پاراگراف جدید استفاده کن، نه Enter.

المان‌های متنی رایج:

المان	معنی	نکته
`<strong>`	اهمیت زیاد	معمولاً **Bold** نمایش داده می‌شود
`<em>`	تأکید	معمولاً *Italic*
`<b>` / `<i>`	فقط ظاهر یا نقش خاص (مثلاً نام محصول، اصطلاح)	منسوخ نیستند، ولی برای «اهمیت» از `strong` و `em` استفاده کن
` `	شکستن خط	فقط برای شعر، آدرس و مواردی که شکستن خط جزو محتواست
`<hr>`	جداکننده‌ی موضوعی	
`<code>`	قطعه کد	
`<pre>`	حفظ فاصله‌ها و Enterها	برای بلوک‌های کد
`<blockquote>`	نقل‌قول بلند	
`<small>`	متن فرعی (مثلاً حقوق کپی‌رایت)	

Entities (نویسه‌های ویژه): برای نمایش خودِ کاراکترهای `<` و `>` و `&` در متن:

```
<p>علامت «و» است &amp;. استفاده کن &lt;p&gt; برای نوشتن تگ پاراگراف از</p>
```

> **ارتباط با فصل‌های بعد:** همین مسئله (تفاوت «متن» و «کد HTML») ریشه‌ی حمله‌ی XSS است (فصل Security).

اشتباهات رایج

- استفاده از `<br><br>` به جای فاصله‌گذاری (کار CSS است).
- انتخاب Heading بر اساس اندازه‌ی ظاهری.
- استفاده از `<b>` به‌جای `<strong>` وقتی منظور «مهم بودن» است.

> **تمرین کوچک:** یک مقاله‌ی کوتاه درباره‌ی خودت بنویس با یک `h1`، دو `h2` و چند پاراگراف. در یکی از پاراگراف‌ها کلمه‌ای را با `strong` و کلمه‌ی دیگر را با `em` مشخص کن.

۲.۴ لینک‌ها [ضروری]

Syntax:

```
<a href="https://example.com">رفتن به سایت</a>
```

انواع `href` :

```
<!-- لینک خارجی (آدرس کامل) -->
<a href="https://developer.mozilla.org">MDN</a>

<!-- لینک نسبی به صفحه‌ای در همین پروژه -->
<a href="about.html">درباره‌ی ما</a>
<a href="pages/contact.html">تماس</a>
<a href="../index.html">بازگشت به خانه</a>

<!-- پرش به بخشی از همین صفحه (با id) -->
<a href="#faq">سؤالات متداول</a>
<h2 id="faq">سؤالات متداول</h2>

<!-- ایمیل و تلفن -->
<a href="mailto:info@example.com">ارسال ایمیل</a>
<a href="tel:+989120000000">تماس تلفنی</a>

<!-- باز شدن در تب جدید -->
<a href="https://example.com" target="_blank" rel="noopener noreferrer">سایت (تب جدید)</a>
```

مسیر مطلق و نسبی:

نوع	مثال	کاربرد
مطلق (Absolute)	`https://site.com/a.html`	سایت‌های دیگر
نسبی (Relative)	`css/style.css` ، `../img/a.png`	فایل‌های داخل پروژه

`../` یعنی «یک پوشه بالاتر برو».

نکات مهم

- `target="_blank"` را با احتیاط استفاده کن؛ تب جدید برای همه خوشایند نیست. مرورگرهای مدرن به‌طور پیش‌فرض رفتار `noopener` را اعمال می‌کنند، ولی نوشتن `rel="noopener noreferrer"` عادت خوبی است.

- متن لینک باید **خودش معنا بدهد:** «مشاهده‌ی قیمت‌ها» به‌جای «اینجا کلیک کنید». (screen readerها لیست لینک‌ها را جدا می‌خوانند.)
- **لینک** برای رفتن به یک آدرس است. اگر کاری انجام می‌دهی (ارسال، باز کردن منو)، المان درست `<button>` است (بخش ۲.۷).

اشتباهات رایج

- استفاده از آدرس فایل لوکال مثل `C:\Users\...` در `href` . فقط روی کامپیوتر خودت کار می‌کند.
- اشتباه در مسیر نسبی (حروف بزرگ و کوچک، پوشه‌ها). در سرور لینوکس `Image.PNG` با `image.png` فرق دارد.
- نداشتن متن در لینکی که فقط یک آیکون دارد (در بخش Accessibility می‌بینی).

> **تمرین کوچک:** دو فایل `index.html` و `about.html` بساز و بین‌شان لینک بده. بعد در `index.html` یک لینک به یک Heading پایین همان صفحه بساز.

۲.۵ تصاویر [ضروری]

:Syntax

```
<img src="images/cat.jpg" alt="گربه‌ی خاکستری روی مبل" width="400" height="300" />
```

Attribute	وظیفه
`src`	آدرس تصویر (الزامی)
`alt`	متن جایگزین (الزامی در عمل)
`width` / `height`	ابعاد ذاتی تصویر، برای جلوگیری از «پرش» صفحه هنگام بارگذاری

`alt` برای چه است؟ (۱) screen reader آن را می‌خواند، (۲) اگر تصویر لود نشد نمایش داده می‌شود، (۳) موتور جستجو از آن می‌فهمد تصویر چیست. اگر تصویر تزئینی محض است، `alt=""` (خالی) بگذار، نه حذف کامل.

فرمت‌ها:

فرمت	مناسب برای
JPG	عکس‌ها
PNG	تصاویر با شفافیت و جزئیات تیز
WebP / AVIF	جایگزین مدرن و سبک‌تر برای JPG و PNG
SVG	آیکن و لوگو (برداری، بدون افت کیفیت)

اشتباهات رایج

- حذف `alt` یا نوشتن `alt="image"` و `alt="عکس"` .
- بارگذاری عکس چندمگابایتی برای نمایش در اندازه‌ی کوچک.
- نوشتن متن مهم (مثل قیمت) داخل تصویر.

> **تمرین کوچک:** دو تصویر در صفحه بگذار: یکی که محتوای مهم دارد (alt توصیفی) و یکی صرفاً تزئینی (`alt=""`).

۲.۶ لیست‌ها [ضروری]

```
 <!-- لیست بدون ترتیب -->
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>

<!-- لیست مرتب -->
<ol>
  <li>نصب VS Code</li>
  <li>ساخت پوشه‌ی پروژه</li>
  <li>اجرای Live Server</li>
</ol>

<!-- لیست توصیفی -->
<dl>
  <dt>DOM</dt>
  <dd>نمایش درختی صفحه در مرورگر</dd>
  <dt>CSSOM</dt>
  <dd>نمایش درختی استایل‌ها</dd>
</dl>
```

لیست تو در تو: زیرلیست باید **داخل** یک `li` باشد، نه مستقیم زیر `ul` :

```
<ul>
  <li>Front-End
    <ul>
      <li>HTML</li>
      <li>CSS</li>
    </ul>
  </li>
  <li>Back-End</li>
</ul>
```

ویژگی‌های `ol` **:** `start="5"` (شروع از ۵)، `reversed` (معکوس)، `type="a"` .

نکته: منوی ناوبری هم از نظر معنایی «لیستی از لینک‌ها» است، پس معمولاً با `ul` ساخته می‌شود (در بخش Semantic می‌بینی).

اشتباهات رایج

- `ul` مستقیم داخل `ul` (بدون `li` والد).
- استفاده از `ul` فقط برای ایجاد تورفتگی.
- استفاده از `<br>` و شماره‌ی دستی به‌جای `ol` .

> **تمرین کوچک:** لیست «مراحل رزرو بلیت» را با `ol` بساز و برای یکی از مراحل یک زیرلیست `ul` اضافه کن.

۲.۷ دکمه‌ها [ضروری]

```
<button type="button">ذخیره</button>
<button type="submit">ارسال</button>
<button type="reset">پاک کردن</button>
<button type="button" disabled>غیرفعال</button>
```

`type`	رفتار
`button`	هیچ کاری نمی‌کند (برای JavaScript)
`submit`	فرم را ارسال می‌کند (پیش‌فرض داخل فرم!)
`reset`	مقدارهای فرم را به حالت اول برمی‌گرداند

تفاوت `a` **و** `button` **:**

	<a>	<button>
هدف	رفتن به یک **آدرس**	انجام یک **عمل**
مثال	«مشاهده‌ی محصول»	«افزودن به سبد»، «باز کردن منو»
فعال‌سازی با کیبورد	Enter	Enter و Space

اشتباه رایج: داخل `form` نوشتن `<button>` بدون `type` و تعجب از اینکه فرم ارسال می‌شود (پیش‌فرض `submit` است). ساخت دکمه با `div` هم ممنوع است، چون focus و کیبورد و screen reader را از دست می‌دهی.

> **تمرین کوچک:** سه دکمه با سه `type` بساز. کدام داخل فرم صفحه را Refresh می‌کند؟ (جواب را با امتحان بفهم.)

۲.۸ کامنت‌ها [ضروری]

```
<!-- این یک کامنت است و نمایش داده نمی‌شود -->
<p>متن قابل‌مشاهده</p>

<!--
  کامنت چندخطی
  برای توضیح بخش بزرگ
-->
```

نکته: کامنت‌ها برای هر کسی که «View Source» بزند قابل‌دیدن‌اند. رمز، توکن و اطلاعات حساس در آن‌ها ننویس. از کامنت برای توضیح «چرا» استفاده کن، نه تکرار «چه».

۲.۹ Attribute، class و id [ضروری]

Attribute اطلاعات اضافه به المان می‌دهد و همیشه در تگ باز می‌آید:

```
<a href="about.html" title="صفحه‌ی درباره‌ی ما" class="link" id="about-
link">درباره</a>
```

Attributeهای سراسری (Global) پرکاربرد:

Attribute	کاربرد
`id`	شناسه‌ی **یکتا**
`class`	گروه‌بندی برای استایل و JS (قابل تکرار)
`title`	راهنمای tooltip (به آن برای اطلاعات حیاتی تکیه نکن)
`lang`	زبان یک بخش از متن
`hidden`	مخفی کردن المان
`tabindex`	کنترل ترتیب Tab (بخش Accessibility)
`data-*`	داده‌ی دلخواه (بخش پیشرفته)

Boolean Attribute: وجودش یعنی «روشن»، نیازی به مقدار نیست: `disabled` ، `required` ، `hidden` ، `checked` .

`class` در برابر `id` :

	`class`	`id`
تکرار در صفحه	مجاز	**ممنوع** (یکتا)
چند مقدار روی یک المان	بله: `class="card featured"`	خیر
کاربرد	استایل‌دهی و انتخاب گروهی	لینک به بخش، اتصال `label` ، انتخاب یک المان خاص در JS

```
<div class="card featured">...</div>
<div class="card">...</div>
<h2 id="pricing">قیمت‌ها</h2>
```

نام‌گذاری: انگلیسی، حروف کوچک، با خط تیره (`product-card`). نام را بر اساس **نقش** بگذار، نه ظاهر: `warning` بهتر از `red-text` .

اشتباهات رایج

- دو المان با `id` یکسان (JS و لینک‌ها رفتار نامعلوم پیدا می‌کنند).
- نوشتن `class="a" class="b"` (فقط اولی اعمال می‌شود. درست: `class="a b"`).
- نام‌گذاری بر اساس ظاهر.

تمرین کوچک: سه `div` با کلاس مشترک `card` بساز و فقط به یکی کلاس دوم `featured` بده.
یک `h2` با `id` بساز و از بالای صفحه به آن لینک بده.

۲.۱۰ div، Inline، Block و span [ضروری]

- **Block:** کل عرض را می‌گیرد و از خط جدید شروع می‌شود: `section` ، `ul` ، `h1`..`h6` ، `p` ، `div` .
- **Inline:** فقط به اندازه‌ی محتوا جا می‌گیرد و در ادامه‌ی خط می‌آید: `em` ، `strong` ، `a` ، `span` ، `img` .

```
<div>این یک بلوک است</div>
<div>این هم یک بلوک است (خط جدید)</div>
<span>این اینلاین است</span> <span>و این هم در همان خط</span>
```

`div` **و** `span` المان‌های **بدون معنا** هستند: `div` برای گروه‌بندی بلوکی و `span` برای گروه‌بندی درون‌خطی، وقتی المان معنادارتری وجود ندارد.

قانون طلایی: اول دنبال المان معنادار بگرد (`article` ، `ul` ، `button` ، `nav` ...). فقط اگر وجود نداشت، سراغ `div` و `span` برو. (دلیلش در بخش متوسط.)

ارتباط با CSS: خاصیت `display` که در فصل CSS می‌آید، همین رفتار Block/Inline را تغییر می‌دهد.

تمرین‌های HTML مقدماتی

ساده

1. ساختار کامل یک سند HTML را بدون نگاه کردن بنویس.
2. یک لیست خرید با `ul` و یک لیست مراحل پخت با `ol` بساز.
3. سه تصویر با `alt` مناسب قرار بده.

متوسط

1. دو صفحه‌ی `index.html` و `about.html` با ناوبری دوطرفه بساز.
2. در یک صفحه‌ی طولانی، فهرست پرش (لینک‌هایی که با `#id` به Headingهای پایین می‌روند) بساز.

چالشی

1. صفحه‌ی خودت را در validator.w3.org بررسی کن و همه‌ی خطاها و هشدارها را رفع کن.

آیا HTML مقدماتی را بلدم؟

سؤالات مفهومی

1. چرا HTML زبان برنامه‌نویسی نیست؟ مثالی بزن که این تفاوت را نشان دهد.
2. اگر `<!DOCTYPE html>` حذف شود چه خطری دارد؟
3. چرا `h3` را فقط به خاطر اندازه‌ی کوچک‌ترش به‌جای `h2` نمی‌گذاریم؟
4. `alt=""` با نداشتن `alt` چه فرقی دارد؟
5. چرا `id` تکراری مشکل‌ساز است؟
6. چرا دکمه‌ی ارسال سبد خرید با `<div onclick>` مشکل دارد؟ (سه دلیل بگو)

تمرین کوتاه (کد را بخوان)

1. خروجی این کد چیست و چه ایرادی دارد؟

```
<ul>
  <li>الف</li>
  <ul>
    <li>ب</li>
  </ul>
</ul>
```

1. این دو پاراگراف در مرورگر چه شکلی دیده می‌شوند؟ چرا؟

```
<p>سلام


      دنیا</p>
<p>سلام<br>دنیا</p>
```

1. در کد زیر چند مشکل می‌بینی؟ (حداقل چهار مورد)

```
<html>
<body>
  <h1>صفحه</h1>
  <h4>بخش</h4>
  <img src="C:\Users\me\photo.jpg">
  <a href="x.html">اینجا کلیک کن</a>
  <p id="a">متن</p><p id="a">متن دوم</p>
</body>
</html>
```

پروژه‌ی سطح HTML مقدماتی: «صفحه‌ی معرفی شخصی چندصفحه‌ای»

- **هدف:** ساخت یک وب‌سایت کوچک ۳ صفحه‌ای (خانه، درباره‌ی من، علاقه‌مندی‌ها) فقط با HTML.
- **امکانات:** ناوبری بین صفحه‌ها در هر صفحه؛ عنوان و معرفی؛ حداقل یک لیست مرتب و یک لیست نامرتب؛ حداقل دو تصویر؛ یک لینک خارجی؛ یک لینک `mailto` ؛ فهرست پرش به بخش‌های همان صفحه در صفحه‌ی طولانی‌تر.
- **مهارت‌ها:** ساختار سند، Headingها، لینک‌های نسبی، تصویر، لیست، `id` .
- **محدودیت‌ها:** بدون CSS و JavaScript؛ بدون `div` و `span` (هنوز لازم نیستند)؛ بدون استفاده از `<br>` برای فاصله‌گذاری.
- **معیار قبولی:**
- هر سه صفحه در validator.w3.org بدون خطا باشند.
- `title` هر صفحه یکتا و توصیفی باشد.
- همه‌ی تصاویر `alt` درست داشته باشند.
- هیچ لینک خرابی نباشد.
- سلسله‌مراتب Headingها در هر صفحه درست باشد.

بخش B: HTML متوسط

۲.۱۱ چرا Semantic HTML مهم است؟ [ضروری]

مفهوم. Semantic یعنی «معنادار». به‌جای اینکه همه‌چیز را با `div` بسازیم، از المانی استفاده می‌کنیم که **نقش** آن بخش را بیان می‌کند.

بدون معنا («div soup»):

```
<div class="top">
  <div class="menu">
    <div class="item"><a href="/">خانه</a></div>
    <div class="item"><a href="/blog">بلاگ و</a></div>
  </div>
</div>
<div class="content">...</div>
<div class="bottom">...</div>
```

با معنا:

```
<header>
  <nav>
    <ul>
      <li><a href="/">خانه</a></li>
      <li><a href="/blog">وبلاگ</a></li>
    </ul>
  </nav>
</header>
<main>...</main>
<footer>...</footer>
```

هر دو در مرورگر شبیه‌اند (بعد از CSS)، پس چرا دومی بهتر است؟

دلیل	توضیح
دسترس‌پذیری	screen readerها با «نقاط عطف» (Landmarks) مثل `nav` و `main` صفحه را مرور می‌کنند. کاربر می‌تواند مستقیم به «محتوای اصلی» بپرد.
رفتار پیش‌فرض	`button` خودش focus و فعال‌سازی با کیبورد دارد؛ `div` ندارد.
SEO	موتور جستجو ساختار و اهمیت بخش‌ها را بهتر می‌فهمد.
خوانایی و نگهداری	`<nav>` از `<div class="menu">` گویاتر است و هم‌تیمی‌ها سریع‌تر می‌فهمند.
ابزارها	حالت خواندن مرورگر، افزونه‌ها و ابزارهای ترجمه از معنا استفاده می‌کنند.

> **ارتباط با فصل‌های بعد:** در فصل Accessibility (همین فصل، بخش پیشرفته) می‌بینی که Semantic HTML اولین و مهم‌ترین ابزار دسترس‌پذیری است.

۲.۱۲ المان‌های ساختاری صفحه [ضروری]

المان	نقش
`<header>`	سربرگ صفحه یا یک بخش (لوگو، عنوان، گاهی ناوبری)
`<nav>`	بلوک **اصلی** لینک‌های ناوبری
`<main>`	محتوای اصلی و منحصربه‌فرد صفحه. **فقط یکی** (قابل‌نمایش) در هر صفحه
`<section>`	بخش موضوعی از محتوا که **معمولاً Heading دارد**
`<article>`	محتوای مستقل که بیرون از صفحه هم معنا دارد (مقاله، خبر، کارت محصول، کامنت)
`<aside>`	محتوای جانبی مرتبط (سایدبار، مطالب مرتبط، تبلیغ)
`<footer>`	پاورقی صفحه یا یک بخش (کپی‌رایت، لینک‌های تماس)
`<figure>` + `<figcaption>`	محتوای مستقل (تصویر، نمودار، کد) همراه با عنوان

یک صفحه‌ی کامل:

```
<body>
  <header>
    <h1>وبلاگ فرانت‌اند</h1>
    <nav aria-label="ناوبری اصلی">
      <ul>
        <li><a href="/">خانه</a></li>
        <li><a href="/articles">مقالات</a></li>
        <li><a href="/about">درباره</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <article>
      <header>
        <h2>یادگیری Flexbox</h2>
        <p>نوشته‌ی <time datetime="2026-09-20">۲۹ شهریور ۱۴۰۵</time></p>
      </header>

      <section>
        <h3>Flexbox چیست؟</h3>
        <p>Flexbox روشی برای چیدمان یک‌بعدی است.</p>
        <figure>
          <img src="flex.png" alt="نمودار محور اصلی و محور متقاطع در Flexbox" />
          <figcaption>محور اصلی و محور متقاطع</figcaption>
        </figure>
      </section>
    </article>

    <aside>
      <h2>مطالب مرتبط</h2>
      <ul>
        <li><a href="/grid">آموزش Grid</a></li>
      </ul>
    </aside>
  </main>

  <footer>
    <p><small>تمام حقوق محفوظ است.</small></p>
  </footer>
</body>
```

توضیح بخش‌به‌بخش:

- `header` اصلی صفحه، عنوان سایت و ناوبری را در بر دارد.
- `main` یک بار آمده و شامل `article` و `aside` است.
- `article` یک `header` مخصوص خودش دارد (عنوان و تاریخ). المان‌های `header` و `footer` می‌توانند داخل `article` یا `section` هم بیایند.

- `time` با `datetime` تاریخ را به شکل قابل‌فهم برای ماشین می‌دهد (فرمت ISO).
- `figure` و `figcaption` تصویر و توضیحش را به هم وصل می‌کنند.

چگونه انتخاب کنم؟ (`section` در برابر `article` در برابر `div`)

سؤال	پاسخ
آیا این بخش مستقل است و اگر جدا کنم در RSS یا صفحه‌ی دیگر هم معنا دارد؟	`article`
آیا بخش موضوعی از سند است و Heading طبیعی دارد؟	`section`
فقط برای گروه‌بندی جهت استایل یا چیدمان است؟	`div`

اشتباهات رایج

- استفاده از `section` به‌جای `div` برای هر گروه‌بندی. اگر Heading نمی‌توانی برایش بنویسی، احتمالاً `div` می‌خواهد.
- بیش از یک `main` قابل‌نمایش.
- هر منو و هر لینک‌دار را `nav` کردن. `nav` برای ناوبری **اصلی** است.
- فکر کردن به اینکه Semantic فقط تغییر نام تگ است. هدف معنا و دسترس‌پذیری است.
- استفاده از `<h1>`..`<h6>` بر اساس اندازه یا فقط برای تزئین.

> **تمرین کوچک:** صفحه‌ی `index.html` پروژه‌ی قبلی را با `header` ، `nav` ، `main` ، `footer` بازنویسی کن. بعد تصمیم بگیر کدام بخش `article` است و کدام `section` .

۲.۱۳ فرم‌ها [ضروری]

مفهوم. فرم راه دریافت اطلاعات از کاربر است. `form` عناصر ورودی را گروه می‌کند تا با هم ارسال شوند.

```
<form action="/register" method="post">
  <!-- فیلدها اینجا -->
</form>
```

Attribute	معنی
`action`	آدرس سروری که داده را دریافت می‌کند
`method`	`get` (داده در آدرس، مناسب جستجو) یا `post` (داده در بدنه‌ی درخواست)

خارج از این جزوه: پردازش داده‌ی ارسالی روی سرور (Back-End). در این جزوه فرم‌ها را در مرورگر و با JavaScript پردازش می‌کنیم (فصل Forms).

یک فرم ثبت‌نام کامل

```
<form action="/register" method="post">
  <fieldset>
    <legend>اطلاعات حساب</legend>

    <div>
      <label for="fullname">نام و نام خانوادگی</label>
      <input type="text" id="fullname" name="fullname" autocomplete="name" required
/>
    </div>

    <div>
      <label for="email">ایمیل</label>
      <input type="email" id="email" name="email" autocomplete="email" required />
    </div>

    <div>
      <label for="password">رمز عبور</label>
      <input type="password" id="password" name="password"
             minlength="8" autocomplete="new-password" required />
    </div>
  </fieldset>

  <div>
    <label for="level">سطح تجربه</label>
    <select id="level" name="level">
      <option value="">انتخاب کنید</option>
      <option value="beginner">مبتدی</option>
      <option value="junior">جونیور</option>
      <option value="senior">سنیور</option>
    </select>
  </div>

  <fieldset>
    <legend>روش آشنایی با ما</legend>
    <label><input type="radio" name="source" value="friend" /> دوستان</label>
    <label><input type="radio" name="source" value="ads" /> تبلیغات</label>
    <label><input type="radio" name="source" value="search" /> جستجو</label>
  </fieldset>

  <div>
    <label for="bio">درباره‌ی شما</label>
    <textarea id="bio" name="bio" rows="4" maxlength="300"></textarea>
  </div>

  <label>
    <input type="checkbox" name="terms" value="yes" required />
    قوانین را می‌پذیرم
  </label>
```

```
  <button type="submit">ثبت نام</button>
</form>
```

توضیح اجزا:

`label` : متن توضیحی برای فیلد. با `for` به `id` فیلد وصل می‌شود یا فیلد را داخل خودش می‌گیرد (مثل checkbox بالا). مزیت‌ها: (۱) کلیک روی متن، فیلد را فعال می‌کند، (۲) screen reader می‌فهمد فیلد چیست. **هر فیلد باید label داشته باشد.**

`input` : پرکاربردترین المان فرم. نوع با `type` تعیین می‌شود:

`type`	کاربرد
`text`	متن کوتاه
`email`	ایمیل (اعتبارسنجی ساده + کیبورد مناسب موبایل)
`password`	رمز (نقطه‌چین)
`number`	عدد (`min` , `max` , `step`)
`tel`	تلفن (کیبورد عددی در موبایل، بدون اعتبارسنجی فرمت)
`url`	آدرس اینترنتی
`date`	انتخاب تاریخ
`search`	جستجو
`checkbox`	چند انتخاب مستقل
`radio`	یک انتخاب از چند گزینه
`file`	آپلود فایل
`range`	اسلایدر
`color`	انتخاب رنگ
`hidden`	داده‌ی پنهان

`name` : اسم فیلد هنگام ارسال. **بدون `name` ، مقدار آن فیلد ارسال نمی‌شود.** فرق `id` و `name` : `id` برای اتصال `label` و استایل/JS است، `name` برای ارسال داده.

`radio` : گزینه‌هایی که یک `name` مشترک دارند یک گروه می‌شوند و فقط یکی انتخاب می‌شود. `value` هر گزینه مقداری است که ارسال می‌شود.

`checkbox` : هر کدام مستقل است. اگر چند checkbox یک گروه باشند، با `name` مشترک می‌توانند چند مقدار بفرستند.

`select` / `option` : لیست کشویی. برای گروه‌بندی گزینه‌ها `optgroup` . برای انتخاب چندتایی `multiple` .

`textarea` : متن چندخطی. مقدار پیش‌فرض بین دو تگ می‌آید (نه `value`).

`fieldset` / `legend` : گروه‌بندی فیلدهای مرتبط (به‌خصوص radio و checkbox) با عنوان گروه. برای دسترس‌پذیری مهم است.

`button type="submit"` : ارسال فرم. تفاوت با `button` : `input type="submit"` می‌تواند محتوای HTML داشته باشد و امروزه رایج‌تر است.

`autocomplete` : به مرورگر می‌گوید فیلد چه نوع داده‌ای است تا بتواند پر کند (، `name` ، `email` ، `tel` ، `username` ، `current-password` ، `new-password`). تجربه‌ی کاربری بهتری می‌دهد.

اشتباهات رایج

- **استفاده از `placeholder` به‌جای `label` .** Placeholder با شروع تایپ ناپدید می‌شود و رنگ کم‌رنگش خوانایی کافی ندارد و جایگزین label نیست.
- فراموش کردن `name` و تعجب از اینکه داده ارسال نمی‌شود.
- radioهای هم‌گروه با `name` متفاوت (هر کدام جداگانه انتخاب می‌شوند).
- `id` تکراری یا `for` که به `id` اشتباه اشاره می‌کند.
- دکمه‌ی خارج از `form` و انتظار ارسال.
- استفاده از `type="text"` برای همه‌چیز. نوع درست، اعتبارسنجی و کیبورد مناسب موبایل می‌دهد.

> **تمرین کوچک:** فرم «ثبت سفارش» بساز: نام، تلفن (`tel`)، شهر (`select`)، نوع ارسال (`radio`)، توضیحات (`textarea`)، و یک checkbox «پیام تأیید ارسال شود».

۲.۱۴ اعتبارسنجی داخلی فرم (Validation) [ضروری]

مرورگرها می‌توانند بدون هیچ JavaScript، ورودی‌ها را بررسی کنند و پیام خطا نشان دهند.

Attribute	معنی	روی کدام نوع‌ها کار می‌کند
`required`	پرکردن اجباری	بیشتر فیلدها
`minlength` / `maxlength`	حداقل / حداکثر طول متن	text، email، password، url، search، tel، `textarea`
`min` / `max`	حداقل / حداکثر مقدار	number، range، date و انواع مشابه
`step`	گام مقدار	number، range، date
`pattern`	الگوی Regex	text، email، password، url، search، tel
`placeholder`	نمونه‌ی راهنما داخل فیلد	فیلدهای متنی
`type`	خودش اعتبارسنجی می‌کند	email، url، number

```
<form>
  <label for="username">نام کاربری (۳ تا ۱۵ حرف انگلیسی یا عدد)</label>
  <input type="text" id="username" name="username"
         required minlength="3" maxlength="15"
         pattern="[A-Za-z0-9]+"
         title="فقط حروف انگلیسی و عدد، بدون فاصله"
         placeholder="مثلاً hamed123" />

  <label for="age">سن</label>
  <input type="number" id="age" name="age" min="18" max="100" step="1" required />

  <label for="site">وبسایت (اختیاری)</label>
  <input type="url" id="site" name="site" placeholder="https://example.com" />

  <button type="submit">ارسال</button>
</form>
```

توضیح:

- `pattern="[A-Za-z0-9]+"` یعنی «یک یا چند حرف انگلیسی یا عدد». الگو به‌طور ضمنی روی **کل مقدار** اعمال می‌شود (نیاز به `^` و `$` نیست).
- `title` متن راهنما را در پیام خطا نشان می‌دهد؛ بنویس الگو چه می‌خواهد.
- اگر فرم نامعتبر باشد، مرورگر ارسال را متوقف و اولین فیلد خطادار را فوکوس می‌کند.

نکات مهم

- `min` و `max` روی `type="text"` اثری ندارند (برای متن از `minlength` / `maxlength` استفاده کن).
- `minlength` فقط وقتی خطا می‌دهد که **کاربر** مقدار را تغییر داده باشد، نه مقدار پیش‌فرض.
- برای خاموش کردن اعتبارسنجی مرورگر روی `form` از `novalidate` استفاده می‌شود (وقتی خودت با JS مدیریت می‌کنی؛ در فصل Forms).
- ظاهر فیلدهای معتبر/نامعتبر را بعداً با `:invalid` , `:valid`) CSS) تنظیم می‌کنیم.

> **هشدار مهم: Client-side validation امنیت نیست.** کاربر می‌تواند HTML را در DevTools عوض کند یا درخواست را مستقیم بفرستد. اعتبارسنجی مرورگر فقط **تجربه‌ی کاربری** است. اعتبارسنجی نهایی همیشه باید روی سرور انجام شود. (بحث کامل در فصل Security. سمت سرور هم **خارج از این جزوه** است.)

اشتباهات رایج

- باور اینکه `required` و `pattern` جلوی ورودی مخرب را می‌گیرند.
- نوشتن Regex پیچیده برای ایمیل. `type="email"` در عمل کافی است (و تأیید واقعی ایمیل فقط با ارسال ایمیل ممکن است).
- پیام خطا ندادن به کاربر درباره‌ی اینکه الگو چیست.
- استفاده از `type="number"` برای شماره‌ی تلفن و کد ملی (آن‌ها «عدد» نیستند و صفر اول حذف می‌شود؛ از `type="tel"` یا `text` با `inputmode="numeric"` استفاده کن).

> **تمرین کوچک:** در فرم ثبت‌نام: رمز حداقل ۸ و حداکثر ۳۰ کاراکتر؛ سن بین ۱۸ و ۶۰؛ کد پستی ۱۰ رقمی با `pattern` . بعد سعی کن فرم را نامعتبر ارسال کنی و پیام‌ها را ببین.

۲.۱۵ جدول (Table) [مفید حرفه‌ای]

برای **داده‌ی جدولی** است، نه برای چیدمان صفحه.

```
<table>
  <caption>مقایسه‌ی سه زبان</caption>
  <thead>
    <tr>
      <th scope="col">زبان</th>
      <th scope="col">نقش</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">HTML</th>
      <td>ساختار</td>
    </tr>
    <tr>
      <th scope="row">CSS</th>
      <td>ظاهر</td>
    </tr>
  </tbody>
</table>
```

- `caption` : عنوان جدول. `thead` / `tbody` / `tfoot` : بخش‌های جدول.
- `th` سلول سرستون یا سطر؛ `scope` به screen reader می‌گوید برای ستون است یا سطر.
- `colspan` و `rowspan` برای ادغام سلول‌ها.
- **اشتباه رایج:** ساخت چیدمان صفحه با جدول (قدیمی و ضدّ دسترس‌پذیری). برای چیدمان از Flexbox و Grid استفاده کن.

تمرین‌های HTML متوسط

ساده

1. ساختار یک وبلاگ را فقط با المان‌های Semantic بنویس (بدون هیچ `div`).
2. برای فرم تماس، هر فیلد را با `label` درست وصل کن و با کلیک روی متن label فعال‌شدن فیلد را تست کن.

متوسط

1. فرم ثبت‌نام را با `fieldset` و `legend` گروه‌بندی کن.
2. یک فرم نظرسنجی بساز با radio، checkbox چندتایی و `select` .

چالشی

1. فرمی بساز که فقط با attributeهای HTML (بدون JS) این قوانین را اعمال کند: نام ۳ تا ۲۰ حرف، سن ۱۸ تا ۶۵، ایمیل معتبر، رمز حداقل ۸ کاراکتر، یک checkbox اجباری. بعد در DevTools یکی از `required`ها را حذف کن و فرم را ارسال کن. چه نتیجه‌ای می‌گیری درباره‌ی امنیت؟

آیا HTML متوسط را بلدم؟

سؤالات مفهومی

1. اگر دو سایت با ظاهر یکسان داشته باشیم، یکی با Semantic و یکی با `div`، چه تفاوت‌هایی برای یک کاربر نابینا خواهد داشت؟
2. چه زمانی `article` و چه زمانی `section` درست‌تر است؟
3. چرا `placeholder` جایگزین `label` نیست؟
4. تفاوت `id` و `name` در فیلد فرم چیست؟
5. اگر سه radio با `name`های متفاوت داشته باشند چه می‌شود؟
6. چرا `required` به‌تنهایی از فرم محافظت نمی‌کند؟

تمرین کوتاه (کد را بخوان)

1. چرا این فرم داده‌ای ارسال نمی‌کند یا ناقص ارسال می‌کند؟ (حداقل سه مشکل)

```
 <form action="/send" method="post">
  <input type="text" placeholder="نام">
  <input type="email" id="mail">
  <label for="email">ایمیل</label>
  <input type="radio" name="a" value="1"> بله
  <input type="radio" name="b" value="2"> خیر
</form>
<button>ارسال</button>
```

1. این دو فیلد چه تفاوتی در رفتار دارند؟

```
 <input type="number" name="phone">
<input type="tel" name="phone">
```

1. این کد در چه شرایطی `required` را رد می‌کند و چه موقع نه؟

```
 <select name="city" required>
  <option value="">انتخاب کنید</option>
  <option value="tehran">تهران</option>
</select>
```

پروژه‌ی سطح HTML متوسط: «صفحه‌ی مقاله و فرم ثبت‌نام»

- **هدف:** ساخت یک وبلاگ دو صفحه‌ای با ساختار کاملاً Semantic و یک فرم ثبت‌نام اعتبارسنجی‌شده.
- **امکانات:**
- صفحه‌ی مقاله: `header` با `main` ، `nav` شامل `article` با چند `section` ، یک `figure` با `figcaption` ، یک `aside` (مطالب مرتبط)، `footer` .
- صفحه‌ی ثبت‌نام: فرم با نام، ایمیل، رمز، تکرار رمز، سطح تجربه (`select`)، علاقه‌مندی‌ها (چند checkbox)، روش آشنایی (radio)، توضیحات (`textarea`) و پذیرش قوانین.
- **مهارت‌ها:** المان‌های Semantic، فرم‌ها، attribute ، `fieldset` ، `label` های اعتبارسنجی، `autocomplete` .
- **محدودیت‌ها:** بدون CSS و بدون JavaScript؛ فقط یک `div` در کل پروژه مجاز است و باید دلیلش را در کامنت بنویسی؛ هیچ `placeholder` نباید جای `label` را بگیرد.
- **معیار قبولی:**
- هر دو صفحه بدون خطا در validator.w3.org.
- فقط یک `main` در هر صفحه و Headingها با سلسله‌مراتب درست.
- همه‌ی فیلدها `label` متصل و `name` دارند.
- فرم با داده‌ی نامعتبر ارسال نمی‌شود و پیام مرورگر دیده می‌شود.
- فقط با کیبورد (Tab، Enter، Space) می‌توانی کل فرم را پر و ارسال کنی.

بخش C: HTML پیشرفته

۲.۱۶ Accessibility (دسترس‌پذیری) [ضروری]

مفهوم. دسترس‌پذیری (به اختصار a11y) یعنی صفحه برای همه قابل‌استفاده باشد: کاربران نابینا (screen reader)، کم‌بینا، کسانی که فقط با کیبورد کار می‌کنند، ناشنوا، کاربران با محدودیت حرکتی، و حتی کسی که روی موبایل زیر آفتاب است. این موضوع فقط «کار اخلاقی» نیست؛ بخشی از کیفیت حرفه‌ای و در بسیاری از کشورها الزام قانونی است.

چهار اصل (POUR): محتوا باید **قابل‌درک (Perceivable)**، **قابل‌استفاده (Operable)**، **قابل‌فهم (Understandable)** و **مقاوم (Robust)** باشد.

> **قانون اول:** تا جایی که المان HTML بومی وجود دارد، از آن استفاده کن. ARIA آخرین راه است، نه اولین.

ساختار صحیح Headingها

- screen reader کاربران را با فهرست Headingها هدایت می‌کند، پس Headingها باید **نقشه‌ی منطقی** صفحه باشند.
- ترتیب را نشکن (`h1` ← `h2` ← `h3`) و فقط برای اندازه انتخاب نکن.
- یک `h1` که موضوع اصلی را بگوید (توصیه‌ی رایج).

متن جایگزین تصاویر (`alt`)

نوع تصویر	`alt` مناسب
اطلاعاتی (عکس محصول، نمودار)	توضیح کوتاه و معنادار: `alt="مدل آبی دویدن کفش X"`
تزئینی (خط جداکننده، پس‌زمینه‌ی تزئینی)	`alt=""` (خالی) تا نادیده گرفته شود
تصویر داخل لینک یا دکمه	**مقصد یا عمل** را بگو: `alt="صفحه‌ی اصلی"` نه «لوگو»
پیچیده (نمودار پرجزئیات)	`alt` کوتاه + توضیح کامل در متن کنار آن

قواعد: «تصویر ...» یا «عکس ...» نگو (screen reader خودش می‌گوید)، و اگر تصویر متن دارد، آن متن را در `alt` بیاور.

دسترسی با کیبورد

بسیاری از کاربران فقط با کیبورد کار می‌کنند. همه‌ی کارهایی که با ماوس ممکن است، باید با `Tab`، `Space`، `Enter`، `Shift+Tab` و Esc ممکن باشد.

- المان‌های **بومی** (`a`، `button`، `input`، `select`، `textarea`) خودکار focus می‌شوند. `div` و `span` نه.
- ترتیب Tab باید با ترتیب منطقی صفحه یکی باشد (ترتیب نوشتن در HTML).
- `tabindex="0"` : المان غیرتعاملی را وارد ترتیب Tab می‌کند (فقط وقتی لازم است).
- `tabindex="-1"` : المان را قابل‌فوکوس با JS می‌کند، ولی از ترتیب Tab خارج می‌کند.
- **هرگز** `tabindex` با مقدار مثبت (مثل `tabindex="3"`) استفاده نکن؛ ترتیب طبیعی را به‌هم می‌ریزد.
- **Focus را نامرئی نکن** (`outline: none` بدون جایگزین ممنوع؛ در CSS می‌بینی).

لینک پرش (Skip Link): به کاربر کیبورد اجازه می‌دهد از منوی طولانی بپرد:

```
<body>
  <a href="#main" class="skip-link">پرش به محتوای اصلی</a>
  <header>...منوی طولانی...</header>
  <main id="main" tabindex="-1">...</main>
</body>
```

(ظاهر `skip-link` را بعداً با CSS تنظیم می‌کنیم؛ معمولاً تا زمان focus مخفی است.)

فرم‌های قابل‌دسترس

علاوه بر `label` و `fieldset`، پیام راهنما و خطا را به فیلد وصل کن:

```
<div>
  <label for="pass">رمز عبور</label>
  <input type="password" id="pass" name="pass"
         aria-describedby="pass-hint pass-error"
         aria-invalid="true" required />
  <p id="pass-hint">حداقل ۸ کاراکتر.</p>
  <p id="pass-error" role="alert">رمز عبور کوتاه است.</p>
</div>
```

توضیح:

- `aria-describedby` به چند `id` اشاره می‌کند؛ screen reader آن متن‌ها را بعد از label می‌خواند.
- `aria-invalid="true"` فیلد را نامعتبر اعلام می‌کند (فقط وقتی واقعاً خطا دارد).
- `role="alert"` باعث می‌شود وقتی متن خطا ظاهر شد، بلافاصله خوانده شود.
- خطا را **فقط با رنگ قرمز** نشان نده؛ متن هم بنویس.

ARIA (در حد لازم) [مفید حرفه‌ای]

ARIA مجموعه‌ی attributeهایی است که به **فناوری‌های کمکی** اطلاعات اضافه می‌دهد. **ARIA رفتار اضافه نمی‌کند**؛ فقط معنا را اعلام می‌کند (رفتار را باید خودت با JS بسازی).

Attribute	کاربرد
`aria-label`	نام دسترس‌پذیر وقتی متن قابل‌مشاهده نداریم (مثلاً دکمه‌ی فقط آیکن)
`aria-labelledby`	نام را از المان دیگری (با `id`) می‌گیرد
`aria-describedby`	توضیح اضافه از المان دیگر
`aria-expanded`	آیا بخش وابسته باز است؟ (`true` / `false`)
`aria-controls`	این المان کدام المان را کنترل می‌کند
`aria-hidden="true"`	المان را از screen reader پنهان می‌کند (فقط برای تزئینات)
`aria-live`	اعلام تغییرات پویا (مثل «۳ نتیجه پیدا شد»)
`role`	تعیین نقش (فقط وقتی المان بومی ندارد)

مثال: دکمه‌ی باز و بسته کردن منو:

```
<button type="button" aria-expanded="false" aria-controls="menu">
  منو
</button>
<ul id="menu" hidden>
  <li><a href="/">خانه</a></li>
  <li><a href="/about">درباره</a></li>
</ul>
```

با JavaScript هنگام کلیک `aria-expanded` و `hidden` را تغییر می‌دهیم (در فصل DOM). **حالت ARIA باید همیشه با وضعیت واقعی هماهنگ باشد.**

دکمه‌ی فقط آیکن:

```
<button type="button" aria-label="بستن پنجره">
  <svg aria-hidden="true" width="16" height="16"><!-- آیکن X --></svg>
</button>
```

اشتباهات رایج

- `<div role="button">` به‌جای `<button>` (باید focus، Enter و Space را هم خودت بسازی؛ بی‌دلیل سخت‌ترش می‌کنی).
- `aria-hidden="true"` روی المان قابل‌فوکوس (کاربر focus می‌کند ولی چیزی نمی‌شنود).
- ARIA اشتباه بدتر از نبودنش است. «ARIA ندارم» بهتر از «ARIA غلط» است.

- فراموش کردن `lang` صفحه.
- تکیه بر رنگ برای انتقال معنا.
- حذف outline فوکوس.

> **تمرین کوچک:** صفحه‌ای که ساخته‌ای را فقط با کیبورد پیمایش کن (ماوس را کنار بگذار). آیا همه‌چیز قابل‌دسترسی است؟ فوکوس همیشه دیده می‌شود؟ ترتیب Tab منطقی است؟ بعد در DevTools (تب Accessibility) یا ابزار Lighthouse امتیاز دسترس‌پذیری را ببین.

۲.۱۷ مبانی SEO و تگ‌های meta [مفید حرفه‌ای]

مفهوم. SEO یعنی کمک به موتورهای جستجو برای فهمیدن صفحه. در سطح HTML، بخش عمده‌ی آن همان Semantic و Accessibility درست است.

```
 <head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>آموزش Flexbox از صفر | وبلاگ فرانت‌اند</title>
  <meta name="description" content="آموزش گام‌به‌گام Flexbox با مثال‌های واقعی برای
چیدمان صفحه." />
  <link rel="canonical" href="https://example.com/flexbox" />
  <link rel="icon" href="/favicon.ico" />

  <!-- Open Graph: ظاهر لینک در شبکه‌های اجتماعی و پیام‌رسان‌ها -->
  <meta property="og:title" content="آموزش Flexbox از صفر" />
  <meta property="og:description" content="آموزش گام‌به‌گام Flexbox با مثال‌های واقعی."
/>
  <meta property="og:image" content="https://example.com/images/flexbox-cover.jpg"
/>
  <meta property="og:url" content="https://example.com/flexbox" />
  <meta property="og:type" content="article" />
</head>
```

تگ	کاربرد
`title`	مهم‌ترین عنصر head؛ یکتا و توصیفی (حدوداً ۵۰ تا ۶۰ کاراکتر معمولاً بدون برش نمایش داده می‌شود)
`meta description`	خلاصه‌ی صفحه در نتایج جستجو (حدود ۱۵۰ تا ۱۶۰ کاراکتر؛ تضمینی برای نمایش نیست)
`viewport`	سازگاری با موبایل (ضروری)
`canonical`	آدرس اصلی، وقتی چند آدرس محتوای یکسان دارند
`og:*`	پیش‌نمایش در اشتراک‌گذاری
`robots` (`noindex`)	جلوگیری از ایندکس صفحه (مثلاً صفحه‌ی ورود)

درباره‌ی `viewport` : مقدار `width=device-width, initial-scale=1` یعنی «عرض صفحه برابر عرض دستگاه». **از `user-scalable=no` یا `maximum-scale=1` استفاده نکن؛** زوم کردن را از کم‌بینایان می‌گیرد.

شاخص‌های SEO که در HTML دست توست: `title` و `description` یکتا برای هر صفحه، Heading درست، `alt` برای تصاویر، متن لینک توصیفی، ساختار Semantic، سرعت، و سازگاری با موبایل.

> **خارج از این جزوه:** SEO فنی عمیق (sitemap، robots.txt، Structured Data/Schema، SSR، رتبه‌بندی). فعلاً همین مبانی کافی است.

اشتباهات رایج

- `title` تکراری در همه‌ی صفحه‌ها؛ `description` خالی.
- تصویر `og:image` با آدرس نسبی (باید آدرس کامل باشد).
- Keyword Stuffing (تکرار بی‌معنی کلمه‌ی کلیدی). مخاطب اصلی خوانندگان‌اند.
- باور اینکه `meta keywords` هنوز تأثیر دارد (موتورهای جستجوی اصلی آن را نادیده می‌گیرند).

> **تمرین کوچک:** برای هر صفحه‌ی پروژه‌ات `title` و `description` یکتا بنویس. آدرس صفحه را در یک پیام‌رسان بفرست (وقتی منتشر شد) و پیش‌نمایش `og:*` را ببین.

۲.۱۸ عناصر چندرسانه‌ای و تصاویر واکنش‌گرا [مفید حرفه‌ای]

`iframe` : تعبیه‌ی صفحه‌ی دیگر

```
<iframe
  src="https://www.openstreetmap.org/export/embed.html?bbox=..."
  title="نقشه‌ی موقعیت دفتر"
  width="600" height="400"
  loading="lazy"
  referrerpolicy="no-referrer"
></iframe>
```

- `title` **الزامی** (برای screen reader).
- `loading="lazy"` بارگذاری تا نزدیک شدن به دید کاربر به تعویق می‌افتد.
- `sandbox` محدودیت امنیتی روی محتوای تعبیه‌شده می‌گذارد (وقتی از منبع نیمه‌معتمد استفاده می‌کنی).
- **امنیت:** هر `iframe` یعنی اجرای محتوای شخص دیگر در صفحه‌ی تو. فقط از منابع معتبر. بعضی سایت‌ها اصلاً اجازه‌ی تعبیه نمی‌دهند (**خارج از این جزوه:** هدرهای سرور).

`audio` و `video`

```
<video controls width="640" poster="cover.jpg" preload="metadata">
  <source src="lesson.webm" type="video/webm" />
  <source src="lesson.mp4" type="video/mp4" />
  <track kind="captions" src="lesson-fa.vtt" srclang="fa" label="فارسی" default />
  مرورگر شما از ویدیو پشتیبانی نمی‌کند.
</video>

<audio controls>
  <source src="podcast.mp3" type="audio/mpeg" />
  مرورگر شما از پخش صوت پشتیبانی نمی‌کند.
</audio>
```

Attribute	معنی
`controls`	نوار کنترل پخش
`poster`	تصویر پیش از پخش
`preload`	`auto` / `metadata` / `none` (چقدر از قبل بارگذاری شود)
`autoplay`	پخش خودکار (مرورگرها عموماً فقط با `muted` اجازه می‌دهند؛ و برای کاربر آزاردهنده است)
`muted`, `loop`	بی‌صدا، تکرار
`<track>`	زیرنویس و توضیح برای ناشنوایان (دسترس‌پذیری)

چند `source` : مرورگر اولین فرمتی را که پشتیبانی می‌کند انتخاب می‌کند.

`source` ، `picture` و تصاویر واکنش‌گرا

مسئله: یک تصویر ۲۰۰۰ پیکسلی روی موبایل بی‌دلیل سنگین است.

راه‌حل ۱: `srcset` و `sizes` (انتخاب اندازه‌ی مناسب):

```
<img
  src="photo-800.jpg"
  srcset="photo-400.jpg 400w, photo-800.jpg 800w, photo-1600.jpg 1600w"
  sizes="(max-width: 600px) 100vw, 50vw"
  alt="کوهستان در غروب"
  width="800" height="533"
  loading="lazy"
/>
```

- `srcset` : لیست نسخه‌ها همراه با عرض واقعی هر فایل (`w`).
- `sizes` : به مرورگر می‌گوید تصویر در صفحه چه عرضی خواهد داشت (روی موبایل کل عرض، در غیر این صورت نصف). مرورگر خودش بهترین فایل را انتخاب می‌کند.

راه‌حل ۲: `picture` (فرمت‌های مدرن و Art Direction):

```
<picture>
  <source srcset="hero.avif" type="image/avif" />
  <source srcset="hero.webp" type="image/webp" />
  <source media="(max-width: 600px)" srcset="hero-mobile.jpg" />
  <img src="hero.jpg" alt="تیم ما در حال کار" width="1200" height="600" />
</picture>
```

- مرورگر اولین `source` مناسب را می‌گیرد؛ اگر هیچ‌کدام نشد، `img` پایانی.
- `img` الزامی است و `alt` را آن می‌دهد.

فرق `srcset` و `picture`: `srcset` یک تصویر را در **اندازه‌های** مختلف عرضه می‌کند؛ `picture` وقتی لازم است **فرمت** یا **کادر** (crop) متفاوت بدهی.

Lazy Loading و جلوگیری از «پرش» صفحه

- `loading="lazy"` برای تصاویر و `iframe`های **پایین‌تر از دید اولیه**.
- برای **تصویر بزرگ بالای صفحه** (hero)، lazy نگذار؛ سریع باید بیاید.
- همیشه `width` و `height` بنویس؛ مرورگر قبل از لود فضا را رزرو می‌کند و صفحه «نمی‌پرد» (Layout Shift).

اشتباهات رایج

- `iframe` بدون `title`.
- `autoplay` با صدا.
- ویدیو بدون زیرنویس برای محتوای آموزشی.
- `lazy` برای تصویر اولین صفحه.
- فایل تصویر بزرگ‌تر از نیاز.
- فراموش کردن `img` داخل `picture`.

> **تمرین کوچک:** یک تصویر را با `picture` و دو فرمت (WebP + JPG) و یک نسخه‌ی موبایل بارگذاری کن. در DevTools تب Network ببین مرورگر کدام فایل را گرفت و با تغییر عرض پنجره چه عوض می‌شود.

۲.۱۹ Data Attribute، template، dialog، details/summary [مفید حرفه‌ای]

`data-*` Attributes

برای نگه‌داشتن اطلاعات دلخواه روی المان (مفید برای JavaScript):

```
<button type="button" class="add-to-cart" data-product-id="42" data-price="250000">
  افزودن به سبد
</button>
```

با JavaScript: `button.dataset.productId` و `button.dataset.price` (نام `data-product-id` به `productId` تبدیل می‌شود؛ در فصل DOM). همچنین می‌توان با CSS انتخابشان کرد.

هشدار: `data-*` برای اطلاعات **غیرحساس** است؛ هر کاربری آن را می‌بیند و می‌تواند تغییر دهد. قیمت واقعی را هیچ‌وقت از آن نخوان و به آن اعتماد نکن.

`template`

قالب HTML که **نمایش داده نمی‌شود** و به‌صورت قالب برای JavaScript نگه داشته می‌شود:

```
<template id="product-card-template">
  <article class="card">
    <h3 class="card__title"></h3>
    <p class="card__price"></p>
  </article>
</template>
```

با JS محتوای آن را کلون و پر می‌کنیم (فصل DOM). مزیت: HTML قالب را از منطق JS جدا نگه می‌دارد.

`dialog` : پنجره‌ی گفتگو (Modal)

```
<button type="button" id="open-btn">باز کردن</button>

<dialog id="confirm-dialog">
  <h2>حذف حساب</h2>
  <p>آیا مطمئن هستید؟</p>
  <form method="dialog">
    <button value="cancel">انصراف</button>
    <button value="confirm">حذف</button>
  </form>
</dialog>

<script>
  const dialog = document.getElementById("confirm-dialog");
  document.getElementById("open-btn").addEventListener("click", () =>
dialog.showModal());
</script>
```

- `showModal()` پنجره را به‌صورت Modal باز می‌کند: پس‌زمینه غیرقابل‌تعامل می‌شود، با Esc بسته می‌شود و فوکوس را مدیریت می‌کند.
- `<form method="dialog">` با کلیک دکمه، پنجره را می‌بندد و `value` دکمه را به‌عنوان `returnValue` می‌دهد.
- **قدیمی در برابر مدرن:** ساختن Modal با `div` و مدیریت دستی فوکوس و Esc، کار سختی است و باگ دسترس‌پذیری زیاد دارد. `dialog` را ترجیح بده.

`details` / `summary` : آکاردئون ساده بدون JS

```
<details>
  <summary>آیا HTML زبان برنامه‌نویسی است؟</summary>
  <p>خیر؛ زبان نشانه‌گذاری است.</p>
</details>
```

- `open` در `details` یعنی پیش‌فرض باز.
- داخل `summary` فقط محتوای عبارتی (متن و عناصر ساده) بگذار.
- گروه‌بندی چند `details` به‌صورت «فقط یکی باز باشد» (با `name` attribute) در مرورگرهای جدید در دسترس است؛ پیش از استفاده، پشتیبانی را در MDN یا caniuse.com بررسی کن. [پیشرفته/اختیاری]

اشتباهات رایج

- گذاشتن داده‌ی حساس یا قیمت واقعی در `data-*` .

- `template` را با «المان مخفی» اشتباه گرفتن: محتوای `template` اصلاً بخشی از صفحه‌ی فعال نیست.
- ساختن Modal با `div` در حالی که `dialog` وجود دارد.
- ندادن نام دسترس‌پذیر به `dialog` (باید Heading یا `aria-label` داشته باشد).

> **تمرین کوچک:** یک بخش FAQ با سه `details` بساز. یک `dialog` تأیید حذف بساز و با دو دکمه‌ی «انصراف» و «حذف» ببند. (کد JS لازم برای باز کردن dialog در همین متن آمده است.)

۲.۲۰ Performance و Best Practices در HTML [مفید حرفه‌ای]

جای `script` و `async` / `defer`

مرورگر هنگام رسیدن به `<script>` معمولاً **پردازش HTML را متوقف** می‌کند تا آن را دانلود و اجرا کند. این دو مشکل می‌سازد: کندی نمایش صفحه، و اجرای اسکریپت **قبل از ساخته شدن** المان‌ها.

> **پاسخ سؤال ۶ فصل ۱:** اگر `<script>` قبل از `<h1>` باشد و کدش `document.getElementById("t")` را صدا بزند، المان هنوز ساخته نشده و نتیجه `null` است؛ استفاده از `.textContent` روی آن خطای `Cannot read properties of null` می‌دهد.

روش	رفتار
`<script src="a.js">` (در head)	HTML متوقف می‌شود، اجرا بلافاصله ← خطر `null`
`<script src="a.js">` (آخر body)	کار می‌کند، ولی دانلود دیرتر شروع می‌شود
`<script src="a.js" defer>`	**دانلود موازی، اجرا بعد از تمام شدن پارس HTML**، به‌ترتیب نوشتن
`<script src="a.js" async>`	دانلود موازی، اجرا **به‌محض آماده‌شدن** (ترتیب تضمین نیست)
`<script type="module">`	رفتاری شبیه `defer` (فصل Modules)

توصیه: برای اسکریپت‌های خودت `defer` را در `head` بگذار:

```
<head>
  <link rel="stylesheet" href="css/style.css" />
  <script src="js/main.js" defer></script>
</head>
```

`async` برای اسکریپت‌های مستقل (مثل آنالیتیکس) مناسب است، نه برای کدی که به ترتیب یا به DOM وابسته است.

قواعد عملکرد پایه

- تصاویر را بهینه (حجم کم، فرمت مدرن، اندازه‌ی مناسب) و `lazy` کن.
- `width` و `height` تصاویر را بنویس.
- CSS را در `head` و JS را با `defer` بارگذاری کن.
- تعداد المان‌های DOM را بی‌دلیل زیاد نکن.
- از فونت‌ها و کتابخانه‌های غیرضروری پرهیز کن.
- (مباحث عمیق‌تر: فصل Performance Basics.)

Best Practices کلی

- **نام تگ و attribute را با حروف کوچک** بنویس؛ مقدار attributeها را داخل `"` بگذار.
- تورفتگی منظم و یکسان (۲ یا ۴ فاصله).
- از `id` برای انتخاب استایل استفاده نکن (در CSS می‌بینی چرا).
- هر صفحه را با validator بررسی کن.
- ابتدا **محتوا و معنا**، سپس ظاهر.
- ساختار را طوری بنویس که **بدون CSS** هم خوانا و منطقی باشد.

مطالب منسوخ؛ جایگزین‌های امروزی

منسوخ	جایگزین
`<center>`	Flexbox ، `text-align` (CSS)
attribute ، `<font>` های `bgcolor` ، `align`	CSS
`<strike>` ، `<big>`	CSS یا `<del>` / `<s>`
`<blink>` ، `<marquee>`	CSS animation (با رعایت دسترس‌پذیری)
`<frameset>` ، `<frame>`	`iframe` یا معماری جدید
`<acronym>`	`<abbr>`
چیدمان با `<table>`	Flexbox و Grid
Modal با `div` و جاوااسکریپت پیچیده	`<dialog>`

۲.۲۱ چک‌لیست مهارتی HTML

با یک بار مرور صادقانه، هر مورد را علامت بزن. اگر نتوانی بدون کمک انجامش بدهی، علامت نزن.

ساختار و پایه [ضروری]

- [ ] ساختار کامل سند را از حفظ می‌نویسم و می‌دانم هر بخش چه می‌کند.
- [ ] تفاوت `head` و `body` و نقش `lang` ، `charset` و `viewport` را می‌دانم.
- [ ] Headingها را بر اساس **معنا** و با سلسله‌مراتب درست استفاده می‌کنم.
- [ ] لینک مطلق و نسبی و پرش با `#id` را درست می‌سازم.
- [ ] `alt` مناسب برای تصاویر اطلاعاتی، تزئینی و لینک‌دار می‌نویسم.
- [ ] تفاوت `class` و `id` و `a` و `button` را می‌توانم توضیح بدهم.

Semantic و فرم [ضروری]

- [ ] چرا Semantic مهم است را با سه دلیل می‌گویم.
- [ ] `section` و `article` و `div` را درست انتخاب می‌کنم.
- [ ] یک فرم کامل با `radio` ، `select` ، `fieldset` ، `label` و `checkbox` می‌سازم.
- [ ] تفاوت `id` و `name` و دلیل اهمیت `name` را می‌دانم.
- [ ] attributeهای اعتبارسنجی را به‌کار می‌برم و می‌دانم چرا امنیت نیست.

پیشرفته [ضروری و مفید]

- [ ] صفحه‌ام را فقط با کیبورد پیمایش می‌کنم و همه‌چیز در دسترس است.
- [ ] می‌دانم ARIA چه موقع لازم است و چرا «اول المان بومی».
- [ ] `viewport`، `description`، `title` و `og:*` را درست می‌نویسم.
- [ ] `lazy`، `srcset`، `picture` و `width` / `height` تصاویر را می‌شناسم.
- [ ] تفاوت `defer` و `async` را توضیح می‌دهم.
- [ ] `template`، `details`، `dialog` و `data-*` را می‌شناسم و جای استفاده‌شان را می‌دانم.

نکته: اگر چند مورد را نتوانستی علامت بزنی، پیش از رفتن به فصل CSS به بخش مربوطه برگرد. تسلط کامل بر ARIA و تمام مباحث اختیاری لازم نیست؛ تمرکز اصلی روی Semantic، فرم، `alt`، و کیبورد باشد.

تمرین‌های HTML پیشرفته

ساده

1. برای یک دکمه‌ی فقط آیکن، نام دسترس‌پذیر بنویس.
2. یک تصویر را با `srcset` در سه اندازه عرضه کن.
3. `defer` را به اسکریپت صفحه اضافه کن و تفاوت را در رفتار ببین.

متوسط

1. یک `dialog` تأیید حذف بساز که با دکمه‌ی «انصراف» و کلید Esc بسته شود.
2. یک ویدیوی آموزشی با `poster` و `track` زیرنویس بگذار.
3. `description`، `title` و `og:*` را برای سه صفحه‌ی متفاوت بنویس.

چالشی

1. با کیبورد و یک screen reader (NVDA در ویندوز، VoiceOver در مک) صفحه‌ات را بخوان و فهرستی از ۵ مشکل پیدا کن و رفع کن.
2. در تب Lighthouse مرورگر Chrome، امتیاز Accessibility و SEO صفحه را بگیر و تا بالای ۹۰ ببر.

آیا HTML پیشرفته را بلدم؟

سؤالات مفهومی

1. چرا `<div role="button">` از `<button>` بدتر است؟ (دست‌کم سه دلیل)
2. چه فرقی بین `alt=""` و `alt` حذف‌شده برای screen reader وجود دارد؟

3. چرا `tabindex="5"` ایده‌ی بدی است؟
4. `aria-hidden="true"` را چه زمانی می‌توان روی یک المان گذاشت و چه زمانی نباید؟
5. تفاوت `srcset` و `picture` چیست؟
6. چرا `data-price` را نباید مبنای محاسبه‌ی قیمت نهایی قرار داد؟
7. تفاوت `defer` و `async` چیست و کدام برای کدی که به DOM وابسته است مناسب‌تر است؟

تمرین کوتاه (کد را بخوان)

1. این فرم چند مشکل دسترس‌پذیری دارد؟ آن‌ها را اصلاح کن.

```
<div class="input">
  <input type="text" placeholder="ایمیل">
  <span style="color:red">خطا</span>
</div>
<div class="btn" onclick="send()">ارسال</div>
<img src="logo.png">
```

1. اگر کاربر روی موبایل با اینترنت کند وارد صفحه شود، کدام قسمت این کد باعث کندی یا پرش صفحه می‌شود؟

```
<head>
  <script src="big-library.js"></script>
</head>
<body>
  <img src="hero-4000px.jpg" loading="lazy">
  <h1>عنوان</h1>
</body>
```

پروژه‌ی سطح HTML پیشرفته: «صفحه‌ی محصول و Landing Page دسترس‌پذیر»

- **هدف:** ساخت دو صفحه (Landing Page یک محصول یا دوره + صفحه‌ی جزئیات محصول) که ساختاری حرفه‌ای، دسترس‌پذیر و آماده‌ی استایل‌دهی داشته باشند. (در فصل CSS همین دو صفحه را استایل می‌دهیم و Dashboard را هم اضافه می‌کنیم.)
- **امکانات:**
- Landing Page: Hero با تصویر واکنش‌گرا، بخش ویژگی‌ها، بخش نظرات کاربران (`article` برای هر نظر)، FAQ با `details`، فرم عضویت در خبرنامه، `footer`.
- صفحه‌ی محصول: گالری تصاویر با `figure`، مشخصات در `table`، دکمه‌ی «افزودن به سبد» با `data-*`، ویدیوی معرفی با زیرنویس، `dialog` برای تأیید افزودن.

- هر دو صفحه: `description` ، `title` و `og:*` اختصاصی؛ Skip Link؛ ناوبری با `aria-label` .
- **مهارت‌ها:** Semantic، فرم‌های قابل‌دسترس، مدیا و تصویر واکنش‌گرا، SEO ، `details` ، `dialog` پایه، `defer` .
- **محدودیت‌ها:** بدون CSS (فقط ساختار) و بدون کتابخانه؛ JavaScript فقط برای باز کردن `dialog` (حداکثر چند خط)؛ هیچ تصویری بدون `alt` مناسب؛ ARIA فقط وقتی المان بومی وجود ندارد و باید دلیلش در کامنت باشد.
- **معیار قبولی:**
- هر دو صفحه بدون خطا در validator.w3.org.
- امتیاز Accessibility در Lighthouse حداقل ۹۰.
- کل صفحه فقط با کیبورد قابل‌استفاده است.
- هیچ Heading خارج از ترتیب نیست و فقط یک `h1` دارد.
- تصاویر زیر دید اولیه `lazy` هستند و همه `width` / `height` دارند.
- بدون CSS هم محتوا منطقی و خوانا خوانده می‌شود.

پاسخ‌نامه‌ی مختصر (پس از تلاش خودت ببین)

HTML مقدماتی

- سؤال ۷: خروجی در مرورگر معمولاً یک لیست با تورفتگی نمایش می‌دهد، ولی ساختار نامعتبر است چون `ul` فقط باید `li` را مستقیم داشته باشد؛ زیرلیست باید داخل `li` باشد.
- سؤال ۸: پاراگراف اول «سلام دنیا» در یک خط (فاصله‌ها و Enterها یکی می‌شوند)؛ دومی در دو خط (به خاطر `br`).
- سؤال ۹: نبود `head/title/charset` ، `lang` ، `DOCTYPE` ؛ پرش از `h1` به `h4` ؛ مسیر محلی `C:\...` ؛ `alt` ندارد؛ متن لینک «اینجا کلیک کن»؛ `id` تکراری.

HTML متوسط

- سؤال ۷: `name` برای فیلدها نیست؛ `label for="email"` ولی `id="mail"` ؛ placeholder به‌جای labelها radio؛ `name` متفاوت دارند؛ دکمه خارج از `form` است.
- سؤال ۸: `number` ورودی را عددی می‌کند (صفر ابتدایی حذف می‌شود، فلش افزایش و کاهش دارد) و برای تلفن مناسب نیست؛ `tel` متن را حفظ می‌کند و روی موبایل کیبورد عددی می‌دهد، ولی فرمت را اعتبارسنجی نمی‌کند.
- سؤال ۹: اگر کاربر گزینه‌ی اول (با `value=""`) را نگه دارد، `required` خطا می‌دهد؛ با انتخاب «تهران» رد می‌شود.

HTML پیشرفته

- سؤال ۲: `alt=""` یعنی «تزئینی است، نادیده بگیر»؛ بدون `alt` ممکن است screen reader نام فایل را بخواند.
- سؤال ۷: `defer` (بعد از پارس HTML اجرا می‌شود و ترتیب حفظ می‌شود).
- سؤال ۸: `placeholder` به‌جای `label`، خطا فقط با رنگ و بدون ارتباط با فیلد، `div` به‌جای `button` (focus و کیبورد ندارد)، تصویر بدون `alt`.
- سؤال ۹: `script` بدون `defer` در `head` رندر را متوقف می‌کند؛ تصویر hero بالای صفحه `lazy` است (باید فوراً بیاید)؛ `width` / `height` ندارد؛ حجم تصویر زیاد است.

فصل ۲ تمام شد. با تأیید تو، فصل ۳ (CSS: مقدماتی، متوسط، پیشرفته) را شروع می‌کنم.